ДИСЦИПЛИНА	Оптимизация программного кода
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Макиевский Станислав Евгеньевич
СЕМЕСТР	7 семестр, 2026-2027 гг.

Практическое занятие 1

Рефакторинг кода

Исходные материалы: студент получает файлы с реализацией (например, `app.js` для Node.js) содержащие проблемный код. В коде умышлено допущены распространенные недочеты дизайна.

Требования/этапы выполнения:

1. Нужно просмотреть исходный код и перечислить найденные проблемы качества. Минимальный набор для обнаружения:
    - Дублирование кода (например, повторяющиеся куски логики в разных функциях).
    - Слишком длинный метод или функции, выполняющие несколько задач.
    - Неверная распределенность ответственности (например, один модуль делает и расчеты, и вывод, и сохранение – нарушение SRP).
    - Непонятные названия переменных или функций.

    Каждую найденную проблему студент должен кратко описать (почему это проблема, на что влияет).
2. Для каждой проблемы – указать, какую технику рефакторинга применить. Например: *“Дублирование кода в функциях A и B – следует вынести общую логику в новую функцию (Extract Function) и вызывать ее в обоих местах”*. Или: “*Функция X слишком длинная – можно разделить на X1, X2 по функциональности (Split Method), например, расчеты отделить от вывода результатов”*. План рефакторинга утверждается преподавателем (можно запросить обратную связь).
3. Реализовать запланированные рефакторинги в исходном коде. Все изменения должны **не** повлиять на работу программы – функционал на выходе тот же.

Следует придерживаться принципа минимальности: изменять постепенно, проверяя после каждого шага, что программа работает (в идеале – запуская тесты, если они есть, или ручное тестирование функциональности).

4. Подготовить краткий отчет (или комментарии в коде), где перечислены выполненные изменения:
    - Какие функции/модули были изменены.
    - Какие методы были добавлены или удалены.
    - Ссылки на шаблоны рефакторинга (из лекции/каталога), которые применены (например, “Extract Method”, “Rename Variable”).
    - Почему код стал лучше: уменьшилось дублирование (указать, сколько строк удалось избежать повторно), повысилась читаемость (объяснить, как именно). Можно включить небольшие **фрагменты кода до и после** для иллюстрации.

      В качестве проверки качества рефакторинга, рекомендуется посчитать цикломатическую сложность выбранной функции до и после. Например: было 5 ветвлений (if/for), M = 6, стало 2 ветвления, M = 3 – заметное снижение. Это продемонстрирует позитивный эффект.